Lesson 13 Teacher's Key: A Hit That Reacts

TIC-80 - Instructor Reference Only

VIDEO GAME CREATION

What This Lesson Teaches

Extending each enemy's table entry with a new 'flash' field, and adding a genuinely new per-frame update loop to manage it - TIC-80's version of GameMaker's Alarm, built from the same plain Lua tools (tables, loops, variables) this engine has used all course, since it has no blocking wait either.

Level 1 - Full Worked Solution

- e.flash = 6 marks a hit enemy for delayed removal instead of removing it immediately - the enemy stays in the table, just in a new "flashing" state.
- The new update loop runs every frame from TIC(), counting down every flashing enemy's and every explosion's timer, removing each once it reaches 0 - this is the engine's substitute for a blocking wait.
- Score, enemies_remaining, and the win-check all still happen immediately in the hit branch, completely unchanged from Lesson 12 - only the actual table.remove() for the enemy is delayed.

Highlighted lines are new this week; everything else is code you already have.

```
-- bullets loop's hit branch (revised):
score = score + 10
enemies_remaining = enemies_remaining - 1
sfx(2)
e.flash = 6
table.insert(explosions, {x=e.x, y=e.y, t=12})
if enemies_remaining <= 0 then
 if wave >= total_waves then
  game_active = false
  has_won = true
  sfx(3)
 else
  wave = wave + 1
  enemies_remaining = 5
  sfx(6)
  spawn_wave()
 end
end

-- new update loop (each frame, before drawing):
for i=#enemies,1,-1 do
```

```
 local e = enemies[i]
 if e.flash then
  e.flash = e.flash - 1
  if e.flash <= 0 then table.remove(enemies, i) end
 end
end
for i=#explosions,1,-1 do
 explosions[i].t = explosions[i].t - 1
 if explosions[i].t <= 0 then table.remove(explosions, i) end
end

-- drawing (per enemy):
if e.flash then spr(FLASH_SPRITE, e.x, e.y, 14) else spr(ENEMY_SPRITE,
e.x, e.y, 14) end
```



Everything in the top box happens the instant the hit is detected. Only the enemy's own removal - the very last step - waits for the pause to finish.

Why This Structure

- Adding a flash field to each enemy's own table entry (rather than a separate parallel table) keeps each enemy's full state - position, alive-ness, now flash status - in one place, the same 'one entity, one table entry' discipline TIC-80 has used since Lesson 4.
- A dedicated explosions table (separate from enemies) is appropriate here because explosions and enemies are genuinely different kinds of things with different lifetimes and different drawing logic - unlike the flash field, which is naturally just an enemy's temporary state.

Level 2 / Level 3 Talking Points

- Ask students to trace what happens to a single enemy's table entry across several frames after being hit: e.flash=6, drawn as the flash sprite, decremented each frame, removed at 0 - a good comprehension check for how table-based state persists across frames.
- For students comparing engines: point out that TIC-80's flash-countdown-in-a-table and GameMaker's alarm[0]-plus-Alarm-event solve the identical problem - both engines lack a blocking wait, so both split 'trigger' and 'finish' into two separate places in code.

Level 2 - complete code:

```
-- Level 2 is verification only - the Level 1 code already produces a
working flash and explosion, so no lines change.
-- Checks: play a full round and confirm every single hit flashes and
explodes, with no enemy left stuck mid-flash and no explosion left on
screen;
-- confirm score/enemies_remaining/wave-clear/win-check all still happen
at exactly the same instant they did before this lesson - only the enemy's
own final removal is delayed;
-- confirm a second bullet hitting an already-flashing enemy does not
double-register a kill.
```

Level 3 - complete code (all three goals combined):

Highlighted lines are the Level 3 additions on top of the Level 1 code.

```
-- new update loop (each frame, before drawing) - Level 3 explosion
animation added:
for i=#enemies,1,-1 do
 local e = enemies[i]
 if e.flash then
  e.flash = e.flash - 1
  if e.flash <= 0 then table.remove(enemies, i) end
 end
end
for i=#explosions,1,-1 do
 local ex = explosions[i]
 ex.t = ex.t - 1
 ex.frame = 1 + math.floor((12 - ex.t) / 4) % 3
 if ex.t <= 0 then table.remove(explosions, i) end
end

-- drawing (per enemy) - Level 3 flicker (replaces the plain flash-vs-
normal check):
if e.flash and e.flash % 2 == 0 then
 spr(FLASH_SPRITE, e.x, e.y, 14)
else
  spr(ENEMY_SPRITE, e.x, e.y, 14)
end

-- drawing (per explosion) - Level 3 multi-frame burst animation:
spr(EXPLOSION_SPRITE + ex.frame, ex.x, ex.y, 14)
```

Common Student Mistakes

- Removing the enemy from the table immediately in the hit branch (old habit from Lesson 4-12), which defeats the whole point - the flash field approach requires NOT removing it there anymore.
- Forgetting to call the new update loop from TIC() every frame, leaving flashing enemies and explosions on screen forever.
- Drawing logic that doesn't check e.flash at all, so hit enemies never actually show the flash sprite even though the field is being set correctly.